The Journal of Supercomputing
https://doi.org/10.1007/s11227-018-2636-7

GPUCloudSim: an extension of CloudSim for modeling and simulation of GPUs in cloud data centers

Ahmad Siavashi[1] · **Mahmoud Momtazpour**[1]

© Springer Science+Business Media, LLC, part of Springer Nature 2018

Abstract

Recent years have witnessed an increasing growth in the usage of GPUs in cloud data centers. It is known that conventional virtualization techniques are not directly applicable to GPUs, making it a challenge to effectively take advantage of virtualization benefits. API remoting, full, para and hardware-assisted virtualization methods are adopted to empower VMs with GPU capabilities. With such a diversity in approaches, there is a need for a simulation environment to study the effectiveness of GPU virtualization techniques and evaluate GPU provisioning and scheduling policies in cloud data centers. In order to model and simulate GPU-enabled VMs in cloud data centers, this work proposes and describes a simulator architecture implemented as an extension of CloudSim. The extension eases up conducting experimental studies that otherwise need to be carried out in real cloud infrastructures. It includes models to simulate interference among co-running applications, the overhead of virtualization and power consumption of GPUs. To demonstrate the usefulness of our extension, we study NVIDIA GRID, a hardware-assisted GPU virtualization solution. We show that for situations where the number of VMs outperforms the number of hosts, the first-fit VM placement of VMware Horizon may not be effective. Instead, we suggest a first-fit increasing VM placement algorithm which increases the acceptance rate by 59%, shortens makespan by 25% and saves energy by 21%.

Keywords Cloud computing · Virtualization · Resource management · Simulation · GPU

✉ Mahmoud Momtazpour
momtazpour@aut.ac.ir; momtazpour@gmail.com

Ahmad Siavashi
siavashi@aut.ac.ir

[1] Department of Computer Engineering and Information Technology, Amirkabir University of Technology, Tehran, Iran

Published online: 16 October 2018

1 Introduction

Cloud computing employs a pay-as-you-go model to offer utility-oriented IT services to end users. Users can request on-demand resources to satisfy their computational requirements. Various services and resources are offered by cloud providers to fit different use cases. Cloud providers use system virtualization to improve resource utilization and reduce total cost of ownership (TCO). Virtualization provides the necessary means to allocate a slice of physical resources to end users.

Graphics processing units (GPUs) are high throughput SIMD processors. The high computational power of GPUs has made them eligible for high-performance computing (HPC) platforms. Figure 1 shows the growth in the number of GPU-equipped supercomputers. Today, cloud providers offer GPU-enabled services for applications in which graphical processors are needed or high computational power is required. Almost all mainstream cloud providers such as Google [2], Amazon [1] and Microsoft [5] have GPU instances to offer.

Simulation is the de facto method of performing experimental research studies on cloud data centers [18]. It is very costly and challenging to have a real controlled cloud environment to evaluate research hypotheses. Therefore, researchers use cloud simulators to perform experimental studies in a repeatable and controllable environment. CloudSim is a very popular cloud simulator in use. It supports the simulation of cloud components such as data centers, hosts, virtual machines (VMs), containers and various resource scheduling and provisioning policies. The extensibility of CloudSim allows researchers to add features that were not previously available in the toolkit [40].

One key challenge to simulate a GPU-enabled cloud data center is the lack of a simulation environment with GPU support. By default, CloudSim is unable to model and simulate GPU-enabled cloud data centers. The purpose of this paper is to provide a simulation framework based on CloudSim to help researches in conducting exper-

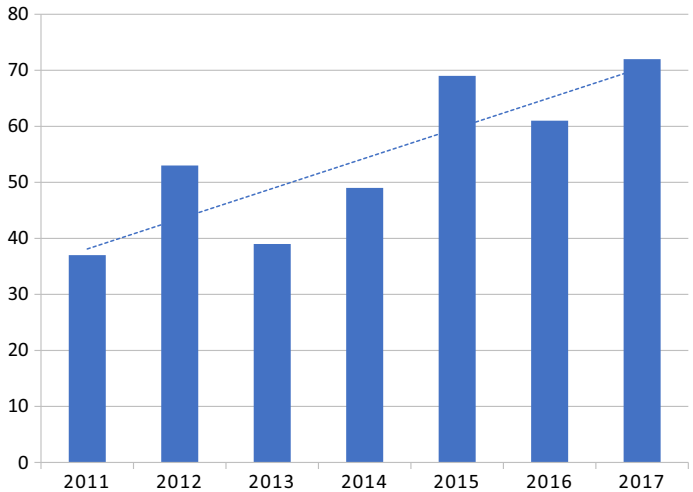


Fig. 1 Number of GPU-equipped supercomputers in the 'Top500' list [7]

imental studies on GPU-enabled cloud data centers. It helps in evaluating the use of different GPU virtualization schemes as well as GPU scheduling and provisioning policies. Such studies will help cloud providers to ensure the effectiveness of the methods they employ and the ultimate satisfaction of end-users. The main contributions of this work can be summarized as follows:

- We provide an extension of CloudSim to model and simulate GPU-enabled data centers.
- We present models to simulate interference among co-running applications, the overhead of virtualization and power consumption of GPUs.
- We enable multi-level scheduling and provisioning of GPU resources.
- We simulate a GPU-enabled cloud data center that employs hardware-assisted GPU virtualization using commercial GPU-enabled VM placement and provisioning algorithms. Then, we propose a more efficient VM placement greedy algorithm.

The rest of the paper is organized as follows: Sect. 2 discusses related work. The scheme of GPU-enabled cloud data centers and the existing GPU virtualization methods are described in Sect. 3. In Sect. 4, the design and implementation details of GPUCloudSim are presented and implemented models and policies that are incorporated in the extension are discussed. Section 5 includes a set of experiments to demonstrate the use of the proposed extension in the implementation of a commercial solution and provides a comparison with proposed algorithms. Section 6 contains final conclusions along with remarks on future research directions. Finally, Sect. 7 provides the necessary data to access the extension.

2 Related work

Simulation makes it possible to conduct experimental researches that are either hard or impossible to perform in the real world. Simulators provide different levels of abstraction and simulated parameters. CloudSim is an extensible event-driven cloud simulator [18]. It provides the ability to simulate cloud computing infrastructures as well as cloud services such as IaaS,[1] PaaS[2] and SaaS[3]. CloudSim modules can be extended to meet user requirements and the available interfaces enable the evaluation of novel provisioning and scheduling policies.

ContainerCloudSim extends CloudSim to incorporate CaaS[4] [40]. Containers provide resource isolation for applications running on a VM. The extension adds support for container allocation, scheduling, and resource provisioning. Moreover, ContainerCloudSim supports power-aware simulation of containerized cloud environments. Modeling and simulation of networked data centers is addressed in [21]

iCanCloud is a cloud simulator with energy-aware simulation support that aims at predicting the trade-offs between cost and performance for a set of applications [35]. For large-scale simulations, iCanCloud is faster than CloudSim but requires more

[1] Infrastructure as a service.

[2] Platform as a service.

[3] Software as a service.

[4] Container as a service.

memory. iCanCloud provides a comprehensive GUI interface which makes it easy to work with the simulator. It also provides a global hypervisor that enables users to examine different provisioning policies.

GreenCloud is a packet-level simulator with an emphasis on energy-aware simulation [30]. The GreenCloud simulator is able to capture energy consumption details of data center components. Furthermore, it supports power saving modes such as DVFS[5] and DNS.[6] Hence, GreenCloud can help in studying and developing energy-efficient resource allocation and workload scheduling solutions.

Lu et al. [32] studied CPU–GPU heterogeneous resource allocation in cloud environments. They proposed an incremental dynamic-adaptive heuristic algorithm which aims at improving resource fairness during the execution of VMs. In order to evaluate the effectiveness of the algorithm and perform large-scale simulations, CloudSim was modified. However, the modifications were made only to suit the particular needs of the research.

GPUs were introduced to address the needs of visual processing. A historical perspective is presented in [9–15] where the first generations of GPU-based systems are involved. Today, almost every computing system leverages the high computational power of GPUs. Moreover, state-of-the-art GPUs can be used for general-purpose computations.

GPGPU-Sim is a well-known and open-source GPU simulator [16]. It provides both functional and cycle-accurate simulation modes. Through GPUWattch, a GPU power simulator, GPGPU-Sim is able to simulate the power consumption of GPU microarchitectural components [31]. Though providing a comprehensive GPU simulation framework, the serial execution of a massively parallel device simulator tends to be approximately 9 orders of magnitude slower than the actual device [44].

Multi2Sim is another mature open-source project that supports cycle-accurate simulation of heterogeneous CPU–GPU systems [43]. The execution of a program on both CPU and GPU is simulated. A program is executed on the CPU simulator, and GPU requests are simulated on the GPU. Beside various CPU and GPU architectures supported by Multi2Sim, recently the NVIDIA Kepler architecture is also added to the framework [22].

The architecture simulation of GPU applications lasts from days to weeks [44]. Therefore, architecture simulators are not applicable for the simulation of cloud environments. On the other hand, cloud simulators mentioned above lack the ability to simulate GPU-enabled entities. Although GPU support for CloudSim is stated in [29], to the best of our knowledge neither the full-text nor the extension is available. Hence, the simulation of GPU-enabled cloud data centers remain a challenge that needs to be addressed.

[5] Dynamic voltage and frequency scaling.

[6] Dynamic network shutdown.

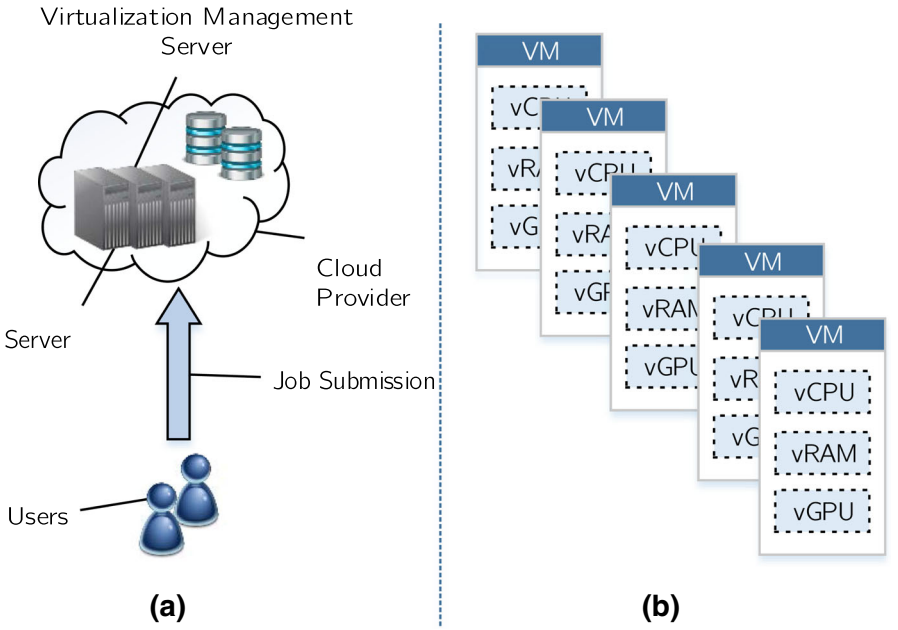


Fig. 2 An overview of a general GPU-enabled cloud computing platform. **a** GPU cloud structure, **b** GPU cloud job

3 GPU-enabled cloud computing

This section describes a general scheme of a GPU-enabled cloud computing environment [27] and then reviews GPU virtualization techniques [25]. Figure 2 shows a simplified topmost view of such a platform. A view of the cloud is shown on the left, while the jobs are shown on the right.

In order to request computational resources, end-users interact with the virtualization management server through a web portal. A job submitted to the cloud is a VM requested by an end-user. Here, VMs can have a vGPU attached. GPU-enabled VMs are assigned to GPU-equipped servers for execution. The servers are connected using an interconnection network.

Figure 3 illustrates main components of GPU-equipped servers and GPU-enabled VMs. A server incorporates one or more CPUs and one, multiple or none video cards. The hypervisor keeps track of all VMs on the server and has the responsibility of virtualizing server resources. This includes resource provisioning and scheduling. A VM requires one or more CPUs and one or none GPU. Moreover, VMs contain end-user applications with tasks that run on CPU and GPU. VMs may experience a performance degradation depending on the virtualization methods used [25].

GPUs reside in video cards where a video card may have multiple GPUs. Video cards are attached to the system using a high-speed PCI Express (PCIe) bus. The full-duplex PCIe bus is shared among onboard GPUs. Copy engines are responsible for transferring data between host and device. A copy engine moves data in one direction

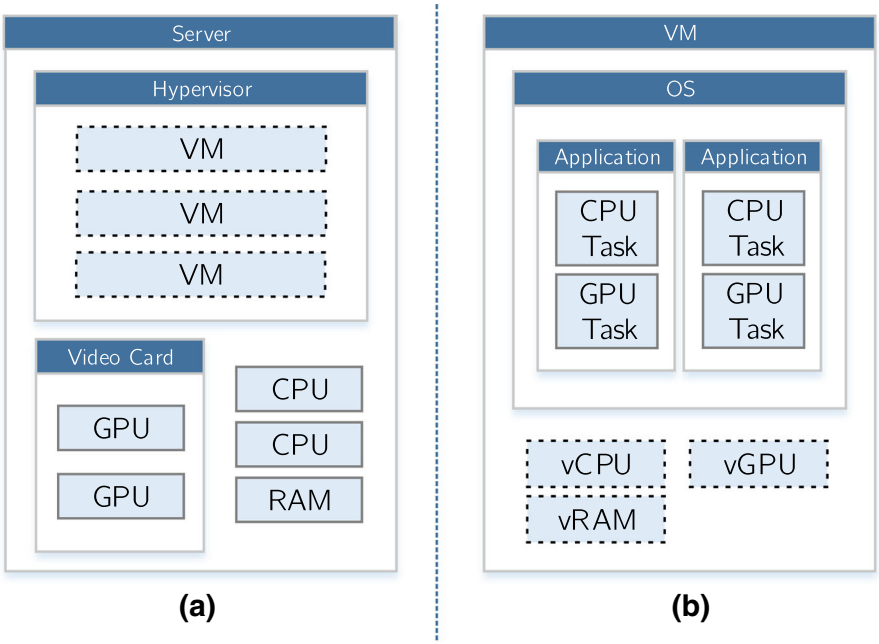


Fig. 3 Components of a GPU-equipped server and a GPU-enabled VM. **a** GPU-equipped server, **b** GPU-enabled VM

at a time. With two copy engines, a GPU is able to simultaneously move data in two distinct directions [19]. Figure 4a shows the components of a dual-GPU video card [36].

A GPU consists of several SIMD[7] cores called SMs.[8] As it is shown in Fig. 4b, to execute a GPU task, the input data must be transferred to the device. Then, the computation is launched by a large number of threads partitioned into a number of blocks. Finally, the output data is transferred back to the host.

The GigaThread engine has the responsibility of scheduling thread blocks on SMs. Thread blocks are distributed equally on SMs and remain there until the end of their execution [16]. It is suggested that GPUs employ cooperative multitasking based on a leftover policy for the scheduling of thread blocks. When multiple applications are running simultaneously on the GPU, they can suffer performance degradation over time. That is due to the interference in accessing shared resources [28].

Virtualization is employed in cloud computing to efficiently use available resources. GPU virtualization is used to share a GPU among multiple VMs. It is employed in various contexts such as HPC and VDI.[9] Due to the lack of vendor support for GPU virtualization, traditionally GPUs were passed through to VMs. However, recently GPU vendors have provided virtualization solutions.

[7] Single instruction multiple data.

[8] Streaming multiprocessor.

[9] Virtual desktop infrastructure.

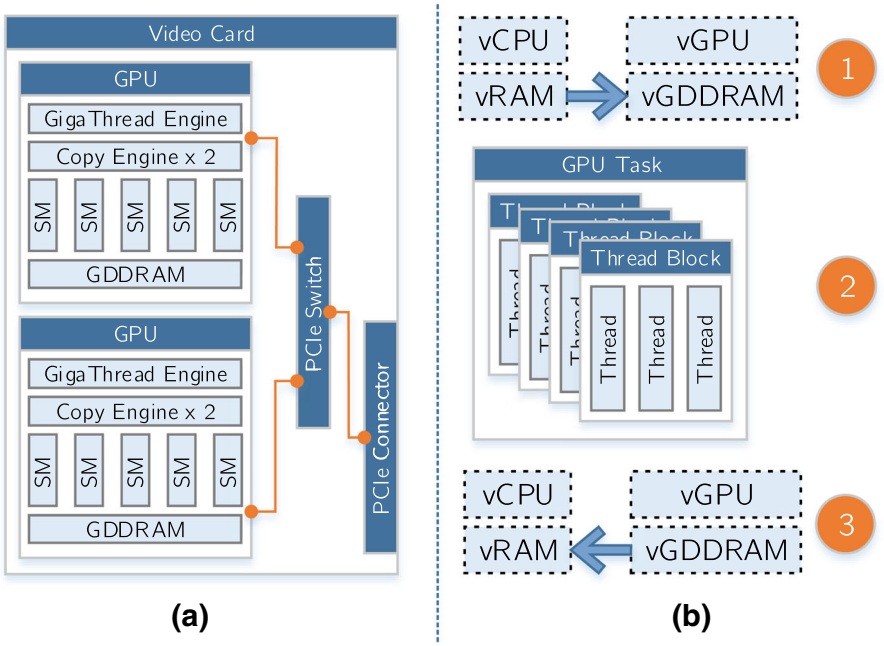


Fig. 4 **a** A dual-GPU video card. A PCIe switch is needed to connect the GPUs to the PCIe bus. Each GPU has its own physically separated GDDRAM (graphics double data rate) memory, **b** execution steps of a GPU task. Firstly, the data is copied to the memory of the GPU. Secondly, the task is executed on the GPU. Finally, the results are copied back to the host. vCPU, vRAM, vGPU, and vGDDRAM are virtualized images of physical CPU, RAM, GPU, and GDDRAM resources

There are several virtualization methods adopted for GPUs with each having their own pros and cons. In the execution stack shown in Fig. 5, *API remoting* approaches virtualize GPUs at the API library level. GPU API calls are intercepted in the guest OS and then forwarded to a GPU-equipped machine. After the request is processed, the result is returned to the caller.

As it is shown in Fig. 5 *full and para virtualization* methods virtualize GPUs at the driver level. In the full virtualization approaches, the vGPU[10] device is generally identical to the physically available GPU. Then, a GPU driver is installed in the guest OS to communicate with the virtualized device. The full virtualization of devices incurs a performance penalty. Therefore, in the para virtualization methods, the guest driver is modified for performance improvement.

In both full and para virtualization methods, the hypervisor is responsible to handle the scheduling of virtualized GPUs. Special-purpose hardware extensions can help in giving direct GPU access to VMs. In *hardware-assisted* approaches, as illustrated in Fig. 5, the hardware is responsible for GPU to VM mappings and multiplexing the GPU between multiple VMs. However, transient hypervisor involvements may be required.

[10] Virtual GPU.

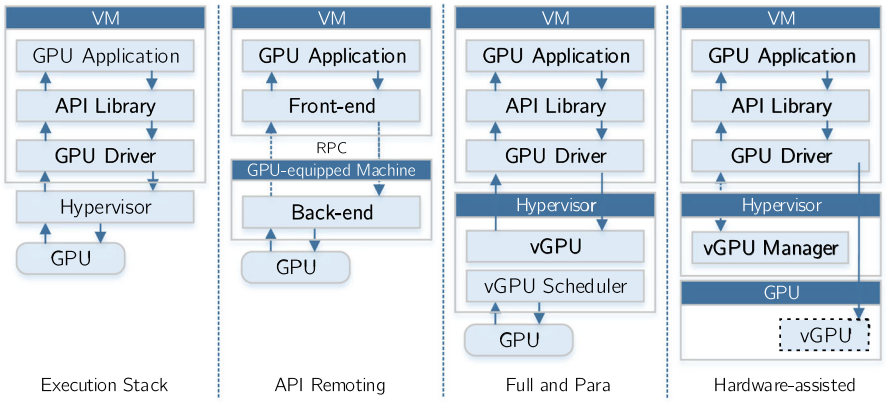


Fig. 5 GPU execution stack and virtualization methods. **a** Space-shared, **b** time-shared, **c** fair-share

4 The proposed GPUCloudSim extension

In order to model and simulate GPU-enabled cloud computing data centers discussed in Sect. 3, we design and implement an extension to CloudSim simulator. CloudSim is among the most popular cloud simulators. It is an extensible event-driven simulator written in Java programming language. As mentioned in Sect. 2, there are packages for simulation of containers, network, power, etc., in CloudSim; however, no package is available for GPU support in cloud data centers. We follow the same architecture of CloudSim, and the proposed extension can be imported without any need for modifications to CloudSim itself. By following the design of CloudSim, the proposed extension enables the researchers to further extend the extension according to their needs. Figure 6 shows the layers of the extension. As it is shown, features are added gradually so that a GPU-enabled cloud computing data center can be simulated based on research requirements.

Firstly, GPUCloudSim layer provides CloudSim with all the necessary classes to model and simulate a GPU-enabled cloud computing data center. Then, Interference-aware GPUCloudSim layer gives GPUCloudSim the ability to model the interference of co-running applications in a GPU. Next, Performance-aware GPUCloudSim layer provides its underlying layer with the ability to model the virtualization-incurred performance degradation of virtualized GPUs. Finally, Power-aware GPUCloudSim layer enables the extension to simulate power-aware GPUs.

In the following, we go through the main classes of the extension. To ease the explanation, we divided the classes according to a top-down approach into three layers, namely cloud layer, server layer, and GPU layer. The class diagrams are generated using ObjectAid UML explorer plugin for Eclipse [6].

4.1 Cloud layer

CloudSim is an event-driven simulator. Figure 7 shows CloudSim and GPUCloudSim main classes that fall into the cloud layer. *SimEntity* is a CloudSim abstract class

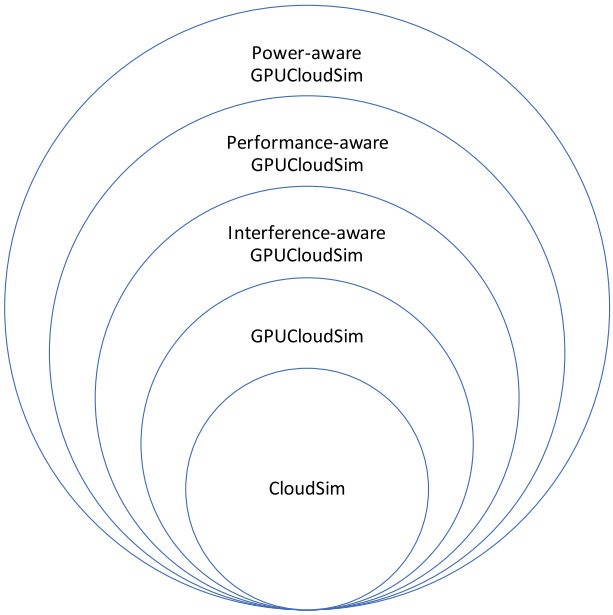


Fig. 6 GPUCloudSim Architecture. Layers are added gradually so that a GPU-enabled cloud computing data center can be simulated based on research requirements. Upper layers add new features to lower layers; hence, lower layers can be used independently from upper layers

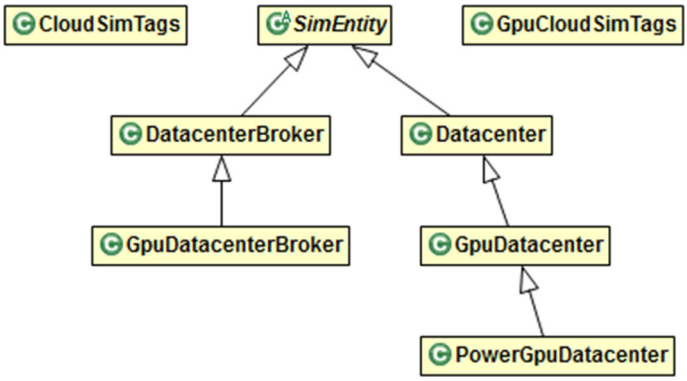


Fig. 7 CloudSim and GPUCloudSim cloud layer class diagram. The **A** superscript denotes abstract classes

extended by event-aware entities within the simulator. *CloudSimTags* contains the events.

DatacenterBroker and *Datacenter* are CloudSim classes that use events to communicate. *DatacenterBroker* acts on behalf of end-users. It queries data centers for allocation of resources and then submits computations. On the other hand, *Datacenter* represents the hardware infrastructure for a data center.

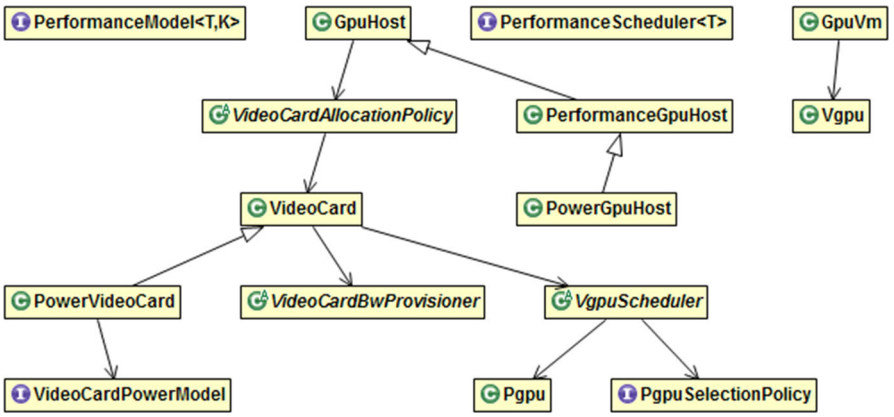


Fig. 8 GPUCloudSim server layer class diagram. The symbol **I** denotes interfaces. Some interfaces are written generically so that they can be used for other entities in future (e.g., PerformanceModel can be used in Host)

GpuDatacenterBroker and *GpuDatacenter* classes are added to support GPU workloads and GPU-equipped hosts, respectively. We consider a GPU job as an end-user GPU-enabled VM containing GPU applications to run. Subsequently, a GPU workload consists of multiple GPU jobs. To distinguish between GPU and non-GPU workloads, *GpuCloudSimTags* provides the necessary events that the two classes need to exchange for GPU workloads.

PowerGpuDatacenter represents a power- and energy-aware GPU-enabled data center. When using this class, a GPU-equipped host has a power model for each video card attached and a power model for the rest of the components. Either *GpuDatacenter* or *PowerGpuDatacenter* can be used based on research requirements.

4.2 Server layer

Figure 8 shows main classes and interfaces for server layer. In order to describe each class and interface, we further divide them according to the layers shown in Fig. 6.

Classes that fall into the GPUCloudSim layer provide basic support for GPU hardware and software in hosts and VMs and include the following:

- GpuVm: It extends CloudSim's *Vm* class to model a VM with a vGPU attached.
- Vgpu: The *Vgpu* class represents a vGPU that is attached to a VM.
- GpuHost: It extends *Host* class from CloudSim to model a GPU-capable host with one or more video cards attached.
- VgpuScheduler: This is an abstract class that defines the policy to share the processing power of a video card among resident vGPUs.
- Pgpu: The *Pgpu* class represents a physical GPU. A *Pgpu* incorporates multiple processing elements that represent GPU SMs.
- VideoCard: This class represents a video card in the system. A video card has one or more *Pgpu* on board, each with its own memory.

- VideoCardAllocationPolicy: It is an abstract class that represents the provisioning policy of video cards to vGPUs. Each *GpuHost* has its own video card allocation policy.
- VideoCardBwProvisioner: This abstract class represents the provisioning policy used by a video card to allocated PCIe bandwidth for its GPUs. Each video card has its own PCIe provisioner.
- PgpuSelectionPolicy: It is an interface implemented by classes that provide a GPU selection policy for vGPUs on a video card. When there are multiple available GPUs for a vGPU on a video card, the selection policy determines which GPU to select.

In the Performance-aware GPUCloudSim layer, to model the virtualization-incurred overhead for vGPUs, which results in performance degradation, new classes and interfaces are needed. The necessary classes and interfaces to simulate this overhead include the following:

- PerformanceGpuHost: This class extends *GpuHost* to support schedulers that model the virtualization overhead for vGPUs. These schedulers implement both *PerformanceScheduler* and *PerformanceModel* interfaces.
- PerformanceScheduler: This is an interface that is implemented by a *VgpuScheduler* with a performance degradation model, in order to become compatible with *PerformanceGpuHost*.
- PerformanceModel: It is an interface that is implemented by a *VgpuScheduler* to model virtualization overheads for vGPUs.

Next, in the Power-aware GPUCloudSim layer, classes are further extended to simulate a power-aware GPU host. The related classes and interfaces include the following:

- PowerGpuHost: This class extends *PerformanceGpuHost* to support power-aware video cards and incorporate a power model for other host components. A power model for the host needs to implement the *PowerModel* interface, provided by the power package of CloudSim.
- PowerVideoCard: It represents a power-aware video card that incorporates a video card power model.
- VideoCardPowerModel: This interface needs to be implemented by the classes that provide a power model for video cards.

The extension includes several implemented scheduling and provisioning policies: *VgpuSchedulerTimeShared*, *VgpuSchedulerSpaceShared* and *VgpuSchedulerFairShare* classes represent GPU provisioning policies based on time-sharing, space-sharing and time-sliced round-robin. Furthermore, the same GPU provisioning policies are available as performance-aware with a *Performance* prefix.

4.3 GPU layer

The class diagram for the GPU layer is shown in Fig. 9. This is the bottom-most layer and represents the finest details of the simulation. Again, in order to describe each class and interface, we further divide them according to the layers shown in Fig. 6.

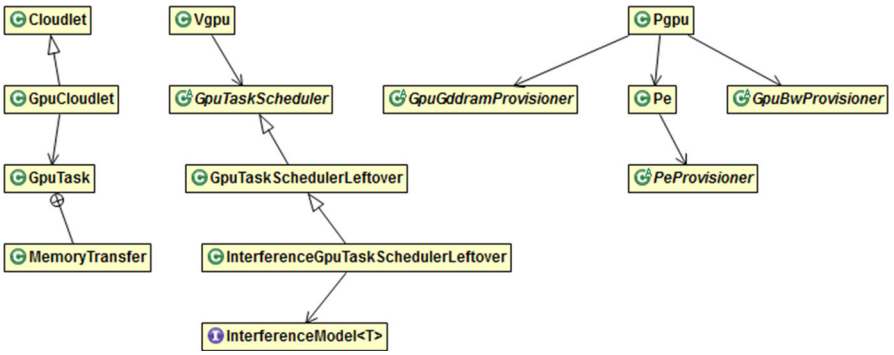


Fig. 9 GPUCloudSim GPU layer class diagram. MemoryTransfer is an inner class of GpuTask. Inheritance facilitates the replacement of a scheduler with another. There is a provisioner for every GPU resource

In the GPUCloudSim layer, which provides the bases for GPU simulation, the GPU layer consists of the following:

- Pe: This is a CloudSim class that represents a processing element. It is used to simulate an SM in the GPU.
- PeProvisioner: This abstract class from CloudSim represents the provisioning policy that is used to allocate the processing power of a *Pe* to vGPUs.
- GpuBwProvisioner: This is an abstract class that represents the provisioning policy of a GPU to allocate GPU memory bandwidth for resident vGPUs.
- GpuGddramProvisioner: This abstract class represents the provisioning policy used by a GPU to allocate GPU memory for resident vGPUs.
- GpuCloudlet: It extends *Cloudlet* class from CloudSim to represent an application with both CPU and GPU portions of execution.
- GpuTask: This class represents a computation that is executed on the GPU. It includes the number of thread blocks and the length of each block. In addition, it incorporates utilization models to imitate resource demands.
- MemoryTransfer: This internal class represents a memory transfer that occurs within a *GpuTask* context.
- GpuTaskScheduler: It is an abstract class that represents the scheduling policy employed by a *Vgpu* to execute its GPU tasks. *GpuTaskSchedulerLeftover* extends this class to implement the cooperative multitasking scheduler based on leftover policy discussed in Sect. 3.

In the Interference-aware GPUCloudSim layer, we model the inter-application interference that may slow down co-running GPU tasks. The main classes and interfaces include the following:

- InterferenceModel: This interface needs to be implemented in order to provide a model for inter-application interference.
- InterferenceGpuTaskSchedulerLeftover It extends the *GpuTaskSchedulerLeftover* class to add support for interference models. Co-running GPU tasks may experience a slow down depending on their utilization of shared resources, such as GPU memory.

The extension includes an implemented interference model. *InterferenceModel-GpuMemory* is based on the GPU memory bandwidth utilization of GPU tasks. It is a fair model which imposes equal slow down on the execution of co-running GPU tasks (see Sect. 4.4.7).

4.4 Models and policies

The extension comes with implemented models and policies. This section discusses models and provisioning and scheduling policies incorporated in the extension that need to be explained in more details. The rest are documented in the extension.

4.4.1 Video card allocation policies

A video card allocation policy is responsible for GPU allocation to vGPUs in a server. As GPUs reside in video cards, the video cards need to be queried for available GPUs in some order. There are three video card allocation policies implemented in the extension. The *Simple* policy is a first-fit policy that iterates over video cards until an available GPU is found. In the *Breadth-first* policy, the cards are traversed in ascending order of their load. In contrast, in the *Depth-first* policy the cards are traversed in descending order of their load.

4.4.2 GPU allocation policies

In a multi-GPU video card, a vGPU can be assigned to multiple available GPUs. The GPU selection policy determines the way in which a GPU is selected among multiple available GPUs in a video card.

There are three implemented GPU allocation policies. The *Simple* policy selects the first available GPU of the card. The *Breadth-first* policy selects the least loaded GPU, whereas the *Depth-first* policy selects the most loaded GPU.

4.4.3 GPU-enabled VM placement policies

A GPU-enabled VM placement policy is responsible for host allocation to VMs in a data center, where the VMs may have vGPUs attached. There are two implemented policies in the extension, namely first-fit (FF) and first-fit increasing (FFI). The latter is proposed in this paper. For a given VM, the FF policy iterates over all hosts until the VM is accepted by a host. On the other hand, the proposed FFI policy first determines the bottleneck resource for each VM-host configurations pair. Then, VMs are sorted in ascending order based on their resource requirements. Finally, VMs are allocated based on FF algorithm. The algorithm for the FFI policy is shown in Algorithm 1.

Algorithm 1: The pseudocode for FFI VM placement.

Input: list of VMs, L_{vm}, list of hosts, L_h, list of host configurations, C_h, list of VM configurations, C_{vm}

Let V be list of bottleneck resource for VM-host configurations pairs;
foreach *vm in* C_{vm} **do**
  foreach *h in* C_h **do**
    Let b be the bottleneck resource for (vm, h);
    $V = V + (vm, h, b)$;
  end
end
// A stable sorting algorithm.
Sort L_{vm} according to V in ascending order;
foreach *vm in* L_{vm} **do**
  foreach *h in* L_h **do**
    if *h accepts vm* **then**
      Use video card and GPU allocation policies to allocate h to vm;
    else
      Add vm at the end of L_{vm};
    end
  end
end

4.4.4 GPU provisioning policies

There are three implemented GPU provisioning policies in the extension, namely space-shared, time-shared and fair-share. Figure 10 illustrates the behavior of provisioning policies for three active vGPUs on a GPU, assuming each vGPU requires half of the computational power of the GPU.

In the *Space-shared* policy, one vGPU occupies the GPU until it is finished. The *Time-shared* policy shares the processing time of GPU among vGPUs. The total processing power of co-running vGPUs cannot exceed that of the GPU. In the *Fair-share* policy, all vGPUs receive a time slice on the GPU. If the total processing power of co-running vGPUs exceeds that of the GPU, the processing power for each vGPU is scaled.

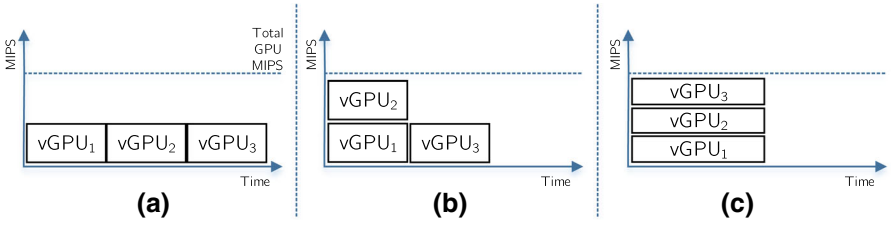


Fig. 10 GPU provisioning policies implemented in GPUCloudSim

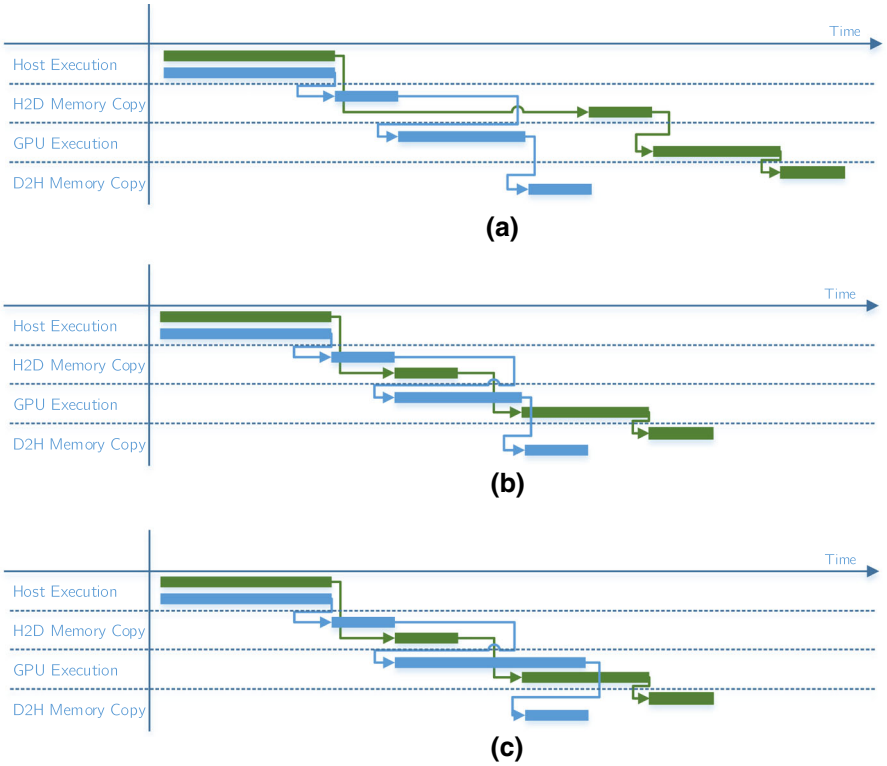


Fig. 11 Leftover policy for the co-execution of two GPU applications on a GPU. **a** GPU resources are completely used by individual tasks, **b** GPU memory is partially used by individual tasks, **c** GPU memory and SMs are partially used by individual tasks

4.4.5 GPU task scheduler

GPU task schedulers are responsible for the provisioning of vGPU resources to GPU tasks. These resources include vGPU's processing power and memory. GPUCloudSim includes a GPU task scheduler based on leftover policy and cooperative multitasking. In this policy, a task occupies the vGPU until it is finished. Co-execution of tasks occurs only if there are enough resources available.

The three execution stages of tasks, namely host to device (H2D) data copy, task execution on GPU and device to host (D2H) data copy may overlap whenever possible [20]. For the co-execution of two GPU applications, Fig. 11a shows a situation in which GPU resources are completely occupied by the first task. Hence, the second task can only progress when the first task releases the GPU. In Fig. 11b the first task leaves enough GPU memory available for the second task to use; however, it occupies all GPU SMs. In this case, the second task performs its H2D memory copy and waits for the release of GPU SMs by the first task. However, there are enough GPU resources for the co-execution of tasks shown in Fig. 11c.

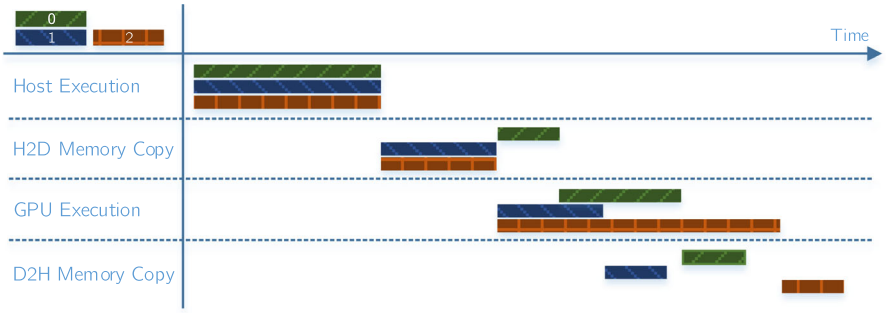


Fig. 12 PCI Express bandwidth provisioning in a dual-GPU video card. The first two applications reside on the first GPU, whereas the third application resides on the second GPU

4.4.6 PCI express bandwidth provisioning

In a multi-GPU video card, onboard GPUs share the full-duplex PCIe bus. As discussed earlier in Sect. 3, each GPU is able to simultaneously have two data transfers in opposite directions. GPUCloudSim implements a PCIe bandwidth provisioning policy in which the bus bandwidth is equally shared among the GPUs that simultaneously send data in the same direction.

Figure 12 illustrates the simultaneous execution of 3 applications on a dual-GPU card. The first two applications run on the first GPU, whereas the third application runs on the second GPU. In the H2D memory copy stage, when the two devices try to simultaneously move data in the same direction, each receives an equal portion of the PCIe bandwidth. Since within each GPU, only one application can move data in one direction, the memory transfer of applications 0 and 1 cannot overlap. In the D2H memory copy stage, application 0 from the first GPU receives the complete PCIe bandwidth as the other GPU has no memory transfer in the same direction. If the two GPUs simultaneously move data in opposite directions, each receives the total bandwidth of PCIe bus.

4.4.7 GPU interference model

Co-execution of GPU tasks on a GPU can lead to performance degradation over time. That is due to the interference in accessing shared resources by co-running tasks. It is shown that simultaneous access to memory is the main contributor to the slow down. Tasks may not experience the same amount of slow down which leads to an unfair execution [28].

GPUCloudSim includes a fair interference model based on memory accesses of tasks. This model imposes equal slow down on co-running tasks. The slowdown experienced by a task can be defined as [28]:

$$\text{Slowdown} = \frac{\text{IPC}_{\text{alone}}}{\text{IPC}_{\text{shared}}} \tag{1}$$

where $\text{IPC}_{\text{alone}}$ is the IPC[11] of a task when executed alone on the GPU and $\text{IPC}_{\text{shared}}$ is the IPC of the task when executed with other tasks. In terms of memory accesses, the slow down can be expressed as:

$$\text{Slowdown} = \frac{\text{Request}_{\text{alone}}}{\text{Request}_{\text{shared}}} \tag{2}$$

where $\text{Request}_{\text{alone}}$ is the number of served memory requests when a task is executed alone on the GPU and $\text{Request}_{\text{shared}}$ is the number of served requests when the task is executed with other tasks. Assuming a direct relationship between requested memory bandwidth by a task in an isolated execution, BW_{alone} and $\text{Request}_{\text{alone}}$ and allocated memory bandwidth to the task when co-executed with other tasks, $\text{BW}_{\text{shared}}$ and $\text{Request}_{\text{shared}}$, the $\text{IPC}_{\text{shared}}$ for a task in simulation is calculated as follows:

$$\text{IPC}_{\text{shared}} = \text{IPC}_{\text{alone}} \times \frac{\text{BW}_{\text{shared}}}{\text{BW}_{\text{alone}}} \tag{3}$$

4.4.8 GPU power model

The power consumption of a GPU depends on various components and parameters. The activity and frequency of components play a significant role in the power consumption of GPUs. It is shown that there is a linear correlation between the power consumption of a GPU and its frequency and utilization [23]. Hence, GPUCloudSim incorporates a linear model to approximate the GPU power. The power model included is as follows:

$$P(f, U) = a_3 fU + a_2 f + a_1 U + a_0 \tag{4}$$

where f and U are the frequency and utilization of SMs and a_is are constants.

5 Evaluation

In order to evaluate the extension and demonstrate its capabilities, we study the use of NVIDIA GRID technology in a cloud computing data center [24]. As a hardware-assisted virtualization technology intended for cloud environments, it enables multiple VMs to have simultaneous direct access to a GPU. In the following, we describe the simulated system. It is assumed the cloud provider leverages VMware Horizon [3] to deliver GPU-enabled services to end-users. Furthermore, it is assumed servers run VMware vSphere [8] as hypervisor and vGPUs are capable of running general-purpose computations.

Figure 13 shows the architecture of GRID vGPU technology. A guest OS loads a vGPU driver which is similar to the driver installed in a non-virtualized environment. The driver then communicates with the GRID Virtual GPU Manager, running in the hypervisor, for a direct access to the physical GPU [39]. The specification for two

[11] Instructions per cycle.

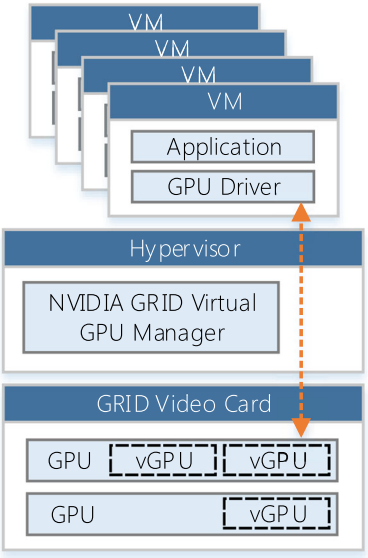


Fig. 13 NVIDIA GRID vGPU architecture

Table 1 The specifications for two GRID graphics cards [37,38]

Card	GPU (s)	Frequency (MHz)	GPU SM (s)	Memory (GB)	TDP[a] (W)
GRID K1	4	850	1	4×4	130
GRID K2	2	750	8	2×4	225

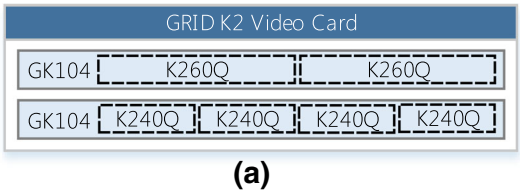
[a]Thermal design power

Table 2 List of vGPU types supported by GRID graphics cards [39]

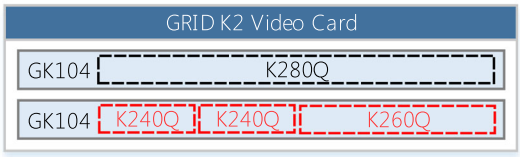
Card	vGPU	Memory (MB)	vGPUs/GPU	vGPUs/Card
GRID K1	GRID K120Q	512	8	32
	GRID K140Q	1024	4	16
	GRID K160Q	2048	2	8
	GRID K180Q	4096	1	4
GRID K2	GRID K220Q	512	8	16
	GRID K240Q	1024	4	8
	GRID K260Q	2048	2	4
	GRID K280Q	4096	1	2

GRID cards is given in Table 1, while the vGPU types supported by each card are listed in Table 2.

In GRID vGPU technology, a physical GPU is either shared among multiple VMs or passed through to a single VM, depending on the vGPU type. The GPU memory is partitioned among resident vGPUs for exclusive access during lifetime; however, GPU SMs are shared among resident vGPUs. For a single physical GPU, at any given time,

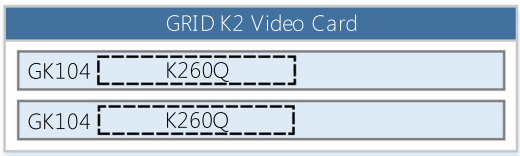


(a)

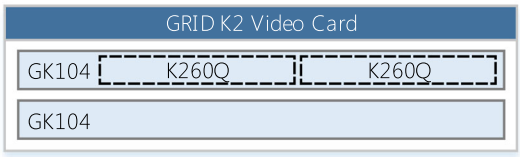


(b)

Fig. 14 Examples for valid and invalid vGPU configurations on a GRID K2 graphics card. A GRID K2 card has two GK104 on-board GPUs. **a** Valid configuration, **b** invalid configuration



(a)



(b)

Fig. 15 Breadth-first and depth-first GPU allocation policies. **a** Breadth-first, **b** depth-first

all resident vGPUs must be of the same type. As an example, Fig. 14 demonstrates valid and invalid configurations on a GRID K2 card.

Horizon employs first-fit policy for VM placement. For every new VM, Horizon starts with the first server and queries vSphere to investigate the possibility of allocating the VM on the server. If the server has no GPU available or the vGPU type is incompatible with the available GPU, the VM is rejected and the next server is tried.

There are two GPU allocation policies employed by vSphere, namely depth-first (DF) and breadth-first (BF) in every server. DF loads up the first available GPU before moving to the next one, whereas BF balances the number of vGPUs on available GPUs. In other words, DF aims at increasing density, whereas BF aims at improving performance. Figure 15 shows an example for DF and BF GPU allocation policies.

In a video card, the provisioning policy explained in Sect. 4.4.6 is employed to equally divide the PCIe bandwidth among onboard GPUs. In a GPU, the fair-share GPU provisioning policy is used to schedule resident vGPUs (see Sect. 4.4.4). Moreover, vGPUs use cooperative multitasking based on leftover policy explained in Sect. 4.4.5, to schedule GPU tasks. The memory interference among co-running GPU tasks is considered (see Sect. 4.4.7) and in cases where the GPU is not passed

Table 3 The list of server configurations based on IBM Bluemix GPU instances

Instance	Frequency (GHz)	Cores	RAM (GB)	Video card	Power (W)
Dual Intel Xeon E5-2620 v3	2.4	12	64	GRID K1	∼330
Dual Intel Xeon E5-2690 v4	2.6	28	128	GRID K2	∼540

The power consumption for each instance is estimated by MSI power supply calculator [4]

Table 4 The list of VM configurations

Instance	Frequency (GHz)	Cores	RAM (GB)	vGPU
1	2.6	8	16	GRID K280Q
2	2.4	4	16	GRID K180Q
3	2.6	4	8	GRID K260Q
4	2.4	2	8	GRID K160Q
5	2.6	2	4	GRID K240Q
6	2.4	2	4	GRID K140Q

through, the virtualization-incurred overhead is assumed to be 10% based on related technologies [26,33,42].

The power models of servers and video cards are considered to have static and dynamic components. In a GPU power model, based on the data available in [31,34,41], the static power is assumed to be 16% of the GPU peak power. In addition, the dynamic part is described by a linear relationship with frequency and utilization (see Sect. 4.4.2). For a server, similar to [17], the static power is considered to be 70% of the server peak power and a linear relationship between dynamic power and utilization is assumed.

The simulation setup includes 5 servers with GRID K1 and 4 servers with GRID K2 graphics cards. The configurations are based on IBM Bluemix GPU instances and are listed in Table 3. 176 VMs are randomly selected from 6 different VM configurations which are based on NVIDIA recommendations. The VM configurations are given in Table 4. 1701 GPU applications are distributed among the VMs where each VM is assigned a random number of applications ranging from 1 to 18. The applications are based on the benchmark introduced in [16]. We profiled the applications and then modeled their behavior in GPUCloudSim. The list of GPU applications is given in Table 5. In order to increase the execution time of applications, the GPU portion of each application is repeated 10^6 times. The VMs are given to the system at the beginning of the simulation.

5.1 Experimental results

Figure 16 shows the normalized average makespan and consumed energy for multiple randomly generated workloads. As it is shown, the difference between BF and DF GPU allocation policies is negligible. This is due to the fact that in situations where there are enough VMs for allocation, the two policies perform similarly. For example, consider the allocation of 4 GRID K260Q VMs on a GRID K2 server. A GRID K2 card

Table 5 The list of benchmark applications and their corresponding properties

	CPU		GPU			
Name	MI[a]	CPU (%)	MI	TB[b]	GPU (%)	Memory BW[c] (%)
BFS	653	High	17	128	Low	Medium
CP	4	Medium	126	256	High	Low
LPS	4	Low	82	100	High	Low
LIB	4	Low	907	64	High	Medium
MUM	1643	High	77	782	Low	Medium
NN	4	High	68	1900	Low	Medium
NQU	4	Low	2	223	Low	Low
RAY	4	High	71	512	High	Medium
STO	4	Low	134	384	High	Low

The utilizations are modeled as normal distributions: $Low \sim \mathcal{N}(0.2, 0.05^2)$, $Medium \sim \mathcal{N}(0.5, 0.05^2)$ and $High \sim \mathcal{N}(0.8, 0.05^2)$

[a] Million instructions

[b] Thread block(s)

[c] Bandwidth

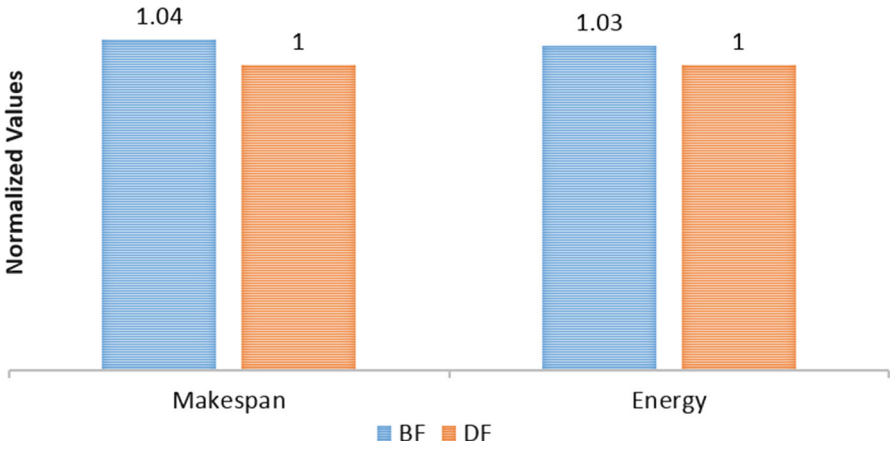


Fig. 16 Simulation results for BF and DF GPU allocation policies. The values are averaged over multiple runs

consists of 2 GPUs where each GPU accepts up to 2 GRID K260Q vGPUs. Although the order of assignments differs for the two policies, the result is the same. Figure 17 shows the order of vGPUs assignments.

Next, we consider the acceptance rate for the two policies. The acceptance rate is defined as the ratio of the number of accepted VMs to the total number of requested VMs at the beginning of the simulation. BF has an acceptance rate of 20%, whereas the number is 22% for DF policy. As DF aims at increasing density, a higher acceptance rate is expected, however, the difference is subtle. Further investigations showed that there are two main reasons to justify the similarity. The first is, as discussed earlier,

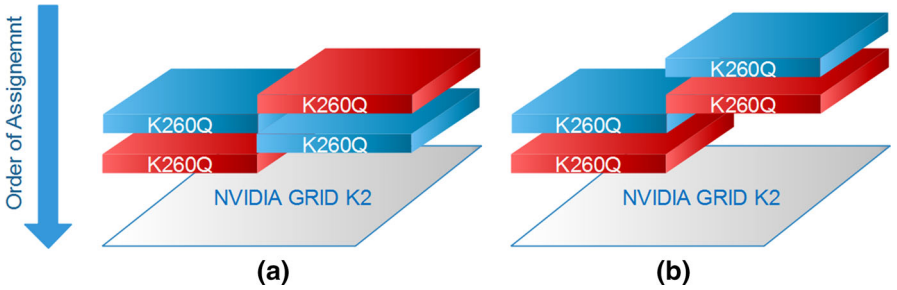


Fig. 17 Order of assignment for four GIRD K260Q vGPUs on a GRID K2 video card, according to BF and DF GPU allocation policies. A GRID K2 card has two GPUs where each GPU accepts up to two GRID K260Q vGPUs

there are enough work for allocation. In such cases, the difference between the two policies is minimized. The second reason is that it is not always the vGPU which prevents VM allocation. In most cases, it is other resources (e.g., CPU) that prevent a VM from being allocated on a server.

In order to study the effects of non-GPU resources, Fig. 18 illustrates a situation in which CPU becomes the bottleneck of VM allocation. With the arrival of the last VM, the server has no more free CPUs to accept more VMs. As both BF and DF policies are not aware of non-GPU resources, their earlier decisions may affect runtime and/or energy consumption. The BF policy aims at increasing performance; however, residing two vGPUs on a GPU would not halve the performance. That is due to the fact that applications may not simultaneously access the GPU. Moreover, the applications tend to access the GPU in different patterns. On the other hand, the DF policy aims at increasing the density of vGPUs on GPUs; however, if the server stops accepting more VMs, idle GPUs would decrease the energy efficiency of the system. Although the idle GPUs contribute to the energy consumption of the server, the accumulated idle energy of the server overcomes that of the GPU.

According to our observations, in order to minimize the makespan and energy consumption of servers, we suggest increasing the load of servers by accommodating more VMs on servers. Furthermore, the experimental results obtained in Sect. 5.1 showed the importance of considering non-GPU resources for the effectiveness of GPU allocation policies. Hence, we accompany the proposed FFI VM placement policy described in Sect. 4.4.3 with the two video card and GPU allocation policies, BF and DF. We call the resulted algorithms FFI-BF and FFI-DF, based on BF and DF allocation policies, respectively. For Video card and GPU allocations, the FFI-BF algorithm employs BF policy, whereas the FFI-DF algorithm uses DF policy.

Figure 19 shows the results for the algorithms. As we discussed earlier, the two first-fit BF and DF policies act similarly at the beginning of the simulation and start to diverge as the number of VMs is decreased. However, as VMs are sorted in FFI-BF and FFI-DF, the difference for the two GPU allocation policies is minimized. In comparison with the 20% and 22% acceptance rate of the first-fit BF and DF policies, FFI-BF and FFI-DF algorithms provide a 35% acceptance rate. The higher acceptance rate provided by the suggested algorithms reduces the VMs waiting time, which leads

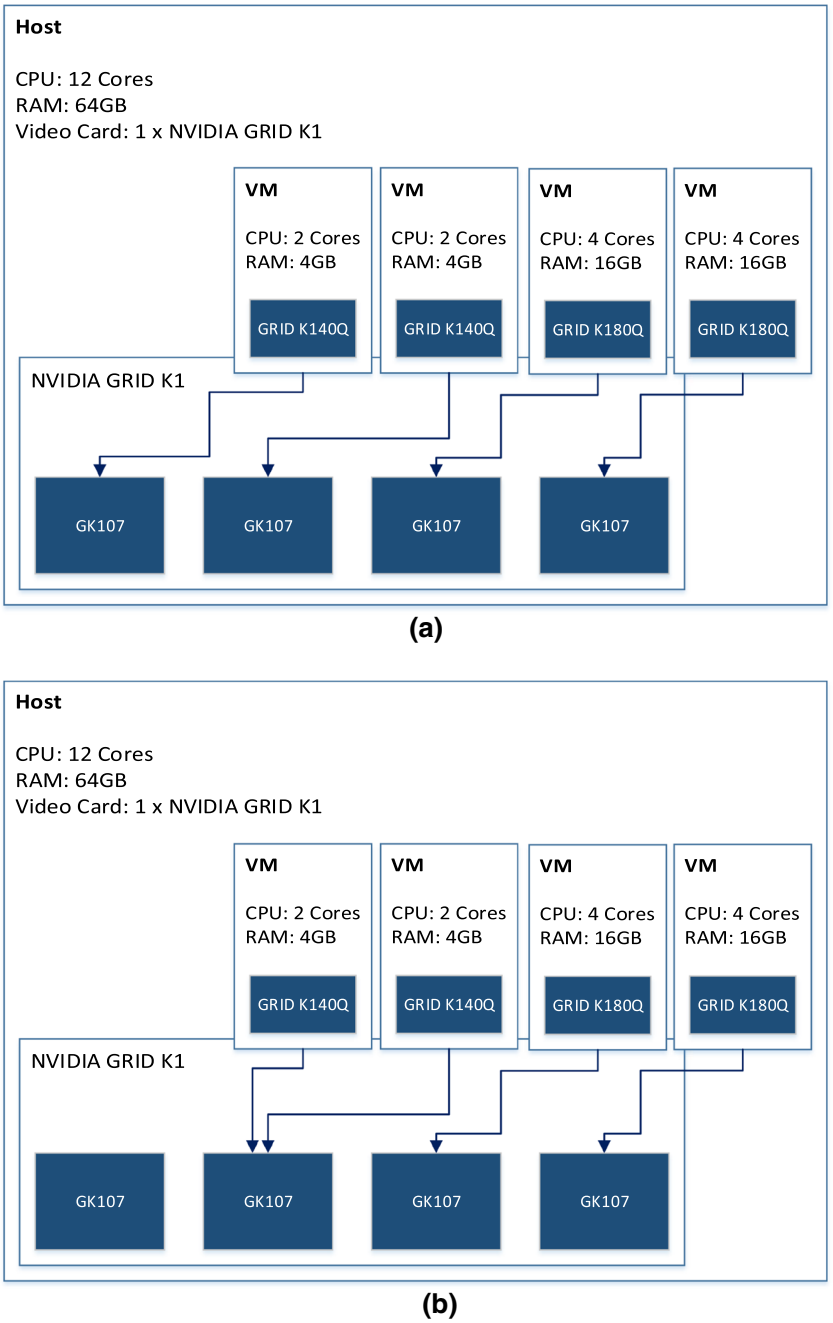


Fig. 18 The behavior of BF and DF GPU allocation policies in a server when non-GPU resources prevent further VMs to be added. BF aims at improving performance while DF aims at increasing density. However, lack of free CPUs prevent further VMs to be added. Hence, previous GPU allocation decisions introduce undesired effects such as inefficient energy consumption or longer execution times. **a** Breadth-first, **b** depth-first

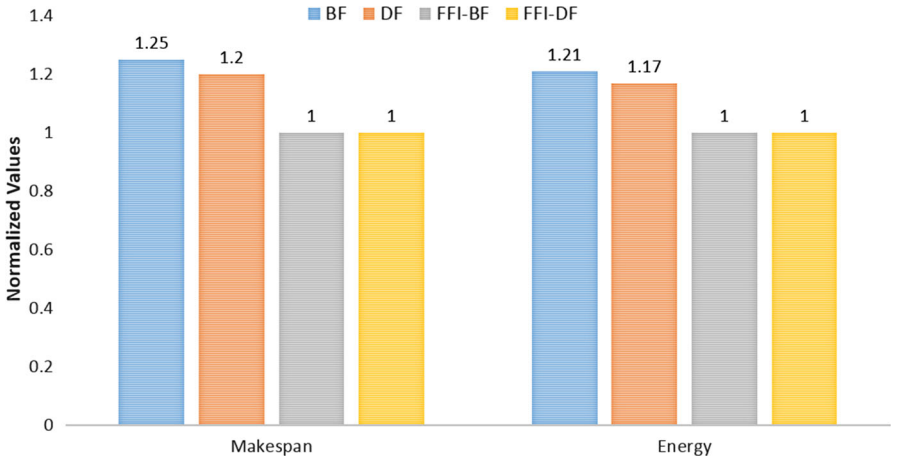


Fig. 19 Comparison of simulation results for first-fit VM placement using BF and DF GPU allocation policies against first-fit increasing VM placement using BF and DF GPU allocation policies

to a reduction in makespan by 25% and 20% for FFI-BF and FFI-DF, respectively. Due to this shorter makespan, FFI-BF and FFI-DF provide 21% and 17% energy savings.

5.2 Scalability

In order to evaluate the scalability of GPUCloudSim, we performed tests on an Intel® Core™ i3-3110M 2.40 GHz machine with 8 GB of RAM. The simulation time complexity depends on algorithms used in both CloudSim and GPUCloudSim components such as models and provisioning and scheduling policies. Hence, the tests are conducted using VMware Horizon policies with BF selected as the GPU allocation policy. In all tests, applications and VMs are equally distributed among VMs and servers, respectively.

Figure 20a shows the scalability of the simulation when the number of entities such as servers, VMs, and applications are equally scaled. In this case, we observed that the simulation time is best described by a quadratic polynomial function of the number of entities rather than a linear function. In Fig. 20b, c an entity is scaled while others are kept fixed. Again, simulation time happened to be best described by a quadratic polynomial function. Figure 20d shows that for a fixed number of VMs and applications, where the number of VMs is greater than that of servers, scaling the number of servers reduces the simulation time.

In order to explain the quadratic time, the internals of the simulator need to be considered. For a data center with m hosts, where each host has v video cards and each video card has p GPUs, the complexity of allocating n GPU-enabled VMs is $O(mnvp)$. In practice, vp is negligible and thus the effective complexity turns to $O(mn)$, which is close to the overhead before adding GPUs. In addition, when scheduling GPU-enabled VMs, the progress of vGPUs is evaluated. Hence, assuming there are c CPU tasks and g GPU tasks running, the complexity of scheduling is $O(mnc+mng) = O(mn(c+g))$. In practice, $c + g$ is negligible, hence turning the complexity to $O(mn)$.

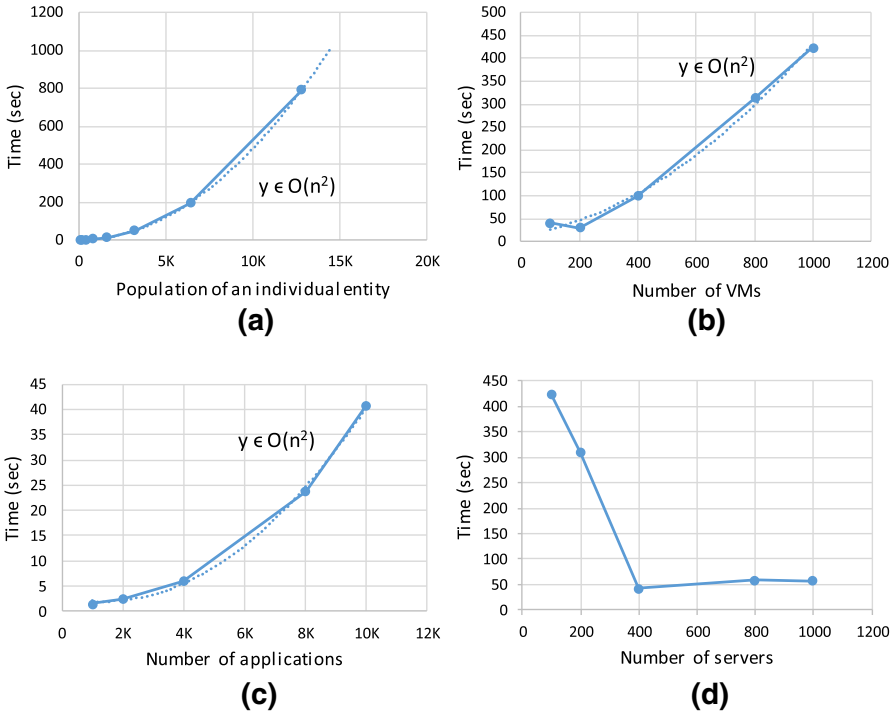


Fig. 20 GPUCloudSim scalability evaluation. **a** Equal number of entities are instantiated, **b** 100 servers, 10,000 applications, **c** 100 servers, 100 VMs, **d** 1000 VMs, 10,000 applications

When the number of simulation entities is equal, instantiation of a 50,000-server data center takes 8 s and almost 120 MB of memory. Our experiments show that GPUCloudSim is scalable enough to simulate modern GPU-enabled cloud computing platforms.

6 Conclusions and future work

In order to satisfy the graphical and computational requirements of end-users, today cloud providers offer GPU-enabled services. State-of-the-art GPUs provide high computational power at the expense of high power consumption. It is known that due to the complexity of GPU devices, conventional virtualization techniques are not directly applicable. Hence, various virtualization methods such as API remoting, full, para and hardware-assisted virtualization techniques are adopted to share a GPU among multiple VMs. To ease up conducting experimental studies on GPU-enabled cloud computing environments, we provided an extension to CloudSim simulator. Our extension included models and provisioning and scheduling policies to enable the modeling and simulation of GPUs in data centers. To demonstrate the usefulness of our extension, we studied NVIDIA GRID, a hardware-assisted GPU virtualization solution. We showed that for situations where the number of VMs outperforms the number of hosts, the first-

fit VM placement of VMware Horizon may not be effective. Instead, we suggested a first-fit increasing VM placement algorithm which would increase the acceptance rate by 59%, shorten makespan by 25% and save energy by 21%. Although high-level simulation may not accurately reflect the behavior of a system, the results are promising and give valuable insights to the use of different policies. In the future, we believe there is a need for studying mixed workloads as well as comparing the effectiveness of different GPU virtualization methods. Moreover, the extension provides the opportunity for studying migration and consolidation techniques on GPU-enabled VMs.

7 Software availability

The source code for GPUCloudSim is publicly available at http://ceit.aut.ac.ir/~lpds. The extension includes several scheduling and provisioning policies, power, performance and interference models and examples to demonstrate how these elements work. The compatibility of the extension has been tested for CloudSim 4.0. To the date of writing, this is the latest version of CloudSim.

References

1. Amazon EC2 instance types—Amazon web services (AWS). https://aws.amazon.com/ec2/instance-types/. Accessed 10 Oct 2018
2. Graphics processing unit (GPU)—Google cloud platform. https://cloud.google.com/gpu/. Accessed 10 Oct 2018
3. Horizon—virtual desktop infrastructure. https://www.vmware.com/products/horizon.html. Accessed 10 Oct 2018
4. MSI global—power supply calculator. https://www.msi.com/power-supply-calculator/. Accessed 10 Oct 2018
5. NVIDIA GPUs in azure: the N-series VMs. http://gpu.azure.com/. Accessed 10 Oct 2018
6. ObjectAid UML explorer. http://www.objectaid.com/. Accessed 10 Oct 2018
7. Top500 supercomputer sites. https://www.top500.org/. Accessed 25 Nov 2017
8. vSphere | server virtualization software. https://www.vmware.com/products/vsphere.html. Accessed 10 Oct 2018
9. Arabnia H (1998) The transputer family of products and their applications in building a high performance computer. In: Belzer J, Holzman AG, Kent A (eds) Encyclopedia of computer science and technology, vol 39. CRC Press, p 283
10. Arabnia H, Oliver M (1989) A transputer network for fast operations on digitised images. Comput Graph Forum 8(1):3–11
11. Arabnia HR (1990) A parallel algorithm for the arbitrary rotation of digitized images using process-and-data-decomposition approach. J Parallel Distrib Comput 10(2):188–192
12. Arabnia HR (1996) Distributed stereo-correlation algorithm. Comput Commun 19(8):707–711
13. Arabnia HR, Bhandarkar SM (1996) Parallel stereocorrelation on a reconfigurable multi-ring network. J Supercomput 10(3):243–269
14. Arabnia HR, Oliver MA (1986) Fast operations on raster images with SIMD machine architectures. Comput Graph Forum 5(3):179–188
15. Arabnia HR, Taha TR (1998) A parallel numerical algorithm on a reconfigurable multi-ring network. Telecommun Syst 10(1–2):185–202
16. Bakhoda A, Yuan GL, Fung WW, Wong H, Aamodt TM (2009) Analyzing CUDA workloads using a detailed GPU simulator. In: International Symposium on Performance Analysis of Systems and Software ISPASS, IEEE, pp 163–174

17. Beloglazov A, Abawajy J, Buyya R (2012) Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. Future Gener Comput Syst 28(5):755–768
18. Calheiros RN, Ranjan R, Beloglazov A, De Rose CA, Buyya R (2011) CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Softw Pract Exp 41(1):23–50
19. Cook S (2012) CUDA programming: a developer's guide to parallel computing with GPUs. Newnes
20. NVIDIA Corporation (2018) CUDA C programming guide. http://docs.nvidia.com/cuda/cuda-c-programming-guide/. Accessed 10 Oct 2018
21. Garg SK, Buyya R (2011) Networkcloudsim: modelling parallel applications in cloud simulations. In: 4th International Conference on Utility and Cloud Computing, IEEE, pp 105–113
22. Gong X, Ubal R, Kaeli D (2017) Multi2Sim Kepler: a detailed architectural GPU simulator. In: International Symposium on Performance Analysis of Systems and Software (ISPASS), IEEE, pp 153–154
23. Guan H, Yao J, Qi Z, Wang R (2015) Energy-efficient SLA guarantees for virtualized GPU in cloud gaming. IEEE Trans Parallel Distrib Syst 26(9):2434–2443
24. Herrera A (2014) NVIDIA GRID: graphics accelerated VDI with the visual performance of a workstation. NVIDIA
25. Hong CH, Spence I, Nikolopoulos DS (2017) GPU virtualization and scheduling methods: a comprehensive survey. ACM Comput Surv (CSUR) 50(3):35
26. Hsu HC, Lee CR (2016) G-KVM: a full GPU virtualization on KVM. In: International Conference on Computer and Information Technology (CIT), IEEE, pp 545–552
27. Hu L, Che X, Xie Z (2013) GPGPU cloud: a paradigm for general purpose computing. Tsinghua Sci Technol 18(1):22–23
28. Hu Q, Shu J, Fan J, Lu Y (2016) Run-time performance estimation and fairness-oriented scheduling policy for concurrent GPGPU applications. In: 45th International Conference on Parallel Processing (ICPP), pp 57–66
29. Jun HW, Hwang CH, Kim K (2014) An extension of CloudSim toolkits for GPGPU-based cloud computing simulation. Information 17:5849–5854
30. Kliazovich D, Bouvry P, Khan SU (2012) GreenCloud: a packet-level simulator of energy-aware cloud computing data centers. J Supercomput 62(3):1263–1283
31. Leng J, Hetherington T, ElTantawy A, Gilani S, Kim NS, Aamodt TM, Reddi VJ (2013) GPUWattch: enabling energy optimizations in GPGPUs. SIGARCH Comput Archit News 41(3):487–498
32. Lu Q, Yao J, Qi Z, He B et al (2016) Fairness-efficiency allocation of CPU-GPU heterogeneous resources. IEEE Trans Serv Comput 1:1
33. Lv Z, Tian K (2014) KVMGT: a full GPU virtualization solution. KVM Forum
34. Mei X, Yung LS, Zhao K, Chu X (2013) A measurement study of GPU DVFS on energy conservation. In: Proceedings of the Workshop on Power-Aware Computing and Systems, ACM, p 10
35. Núñez A, Vázquez-Poletti JL, Caminero AC, Castañé GG, Carretero J, Llorente IM (2012) iCanCloud: a flexible and scalable cloud infrastructure simulator. J Grid Comput 10(1):185–209
36. NVIDIA's next generation CUDA compute architecture: Kepler GK110 Whitepaper, NVIDIA Corporation (2012)
37. NVIDIA Corporation, NVIDIA GRID K1 graphics board (2013)
38. NVIDIA Corporation, NVIDIA GRID K2 graphics board (2013)
39. NVIDIA Corporation, GRID virtual GPU—user guide (2016)
40. Piraghaj SF, Dastjerdi AV, Calheiros RN, Buyya R (2017) ContainerCloudSim: an environment for modeling and simulation of containers in cloud data centers. Softw Pract Exp 47(4):505–521
41. Qouneh A, Liu M, Li T (2015) Optimization of resource allocation and energy efficiency in heterogeneous cloud data centers. In: 44th International Conference on Parallel Processing (ICPP), IEEE, pp 1–10
42. Tian K, Dong Y, Cowperthwaite D (2014) A full GPU virtualization solution with mediated pass-through. In: USENIX Annual Technical Conference, pp 121–132
43. Ubal R, Jang B, Mistry P, Schaa D, Kaeli D (2012) Multi2Sim: a simulation framework for CPU–GPU computing. In: 21st International Conference on Parallel Architectures and Compilation Techniques (PACT), IEEE pp 335–344
44. Yu Z, Eeckhout L, Goswami N, Li T, John L, Jin H, Xu C (2013) Accelerating GPGPU architecture simulation. SIGMETRICS Perform Eval Rev 41(1):331–332